

DevOps Laboratory

Experiments 1 to 8

Experiment 1	Implementation of DevOps related Linux commands
Experiment 2	Staging, Commit and Push on GitHub
Experiment 3	Commit, Modification on Branch Master, Forking and GitHub Repository Link
Experiment 4	CI/CD with Jenkins and Java & cloning in GitHub
Experiment 5	Implementation of Docker commands
Experiment 6	Implementation of Containerization using Docker
Experiment 7	Demonstrate Maven Build Life Cycle
Experiment 8	Install Selenium tool and Automated Testing using Selenium

Experiment No. 1

Title: Implementation of DevOps related Linux commands

Objectives

To implement various Linux commands related to DevOps.

Theory

Linux is an open-source OS widely used in DevOps for server management, automation, and deployment. Linux commands enable file management, process control, package installation, text processing, and system monitoring — all essential DevOps skills. Commands like grep, awk, and pipelines allow efficient log analysis and automation in CI/CD workflows.

Commands and Steps

1. mkdir — Create Directories

Step 1: Create single folder: `mkdir myfolder`

Step 2: Create multiple folders: `mkdir f1 f2 f3`

Step 3: Create nested folder: `mkdir f1/subfolder`

■■ Parent folder must exist before creating nested. If not, use: `mkdir -p f1/sub`

2. ls — List Files and Folders

Step 1: List current directory: `ls`

Step 2: With permissions: `ls -l`

Step 3: Show hidden files: `ls -a`

Step 4: List subfolders recursively: `ls -r`

3. cd — Change Directory

Step 1: Enter folder: `cd foldername`

Step 2: Go back one level: `cd ..`

Step 3: Go to home directory: `cd`

4. touch / echo / cat — Create and View Files

Step 1: Create empty file: `touch myfile.txt`

Step 2: Create file with data: `echo "Hello DevOps" > myfile.txt`

Step 3: View file content: `cat myfile.txt`

Step 4: Append data: `echo "Line2" >> myfile.txt`

Step 5: Open in editor: `nano myfile.txt` — edit, then press `Ctrl+X` → `Y` → `Enter` to save

Step 6: Append interactively: `cat >> myfile.txt` — type text, press `Ctrl+D` to stop

■■ `>` overwrites a file. `>>` appends. Never mix them up.

5. cp — Copy Files and Folders

Step 1: Copy a file: `cp /home/ubuntu/t1/t13 /home/ubuntu/test1/`

Step 2: Copy a folder (use `-r`): `cp -r /home/ubuntu/f1/f13 /home/ubuntu/test1/`

■■ Always use `-r` flag when copying directories, not files.

6. mv — Move (Cut-Paste)

Step 1: Move a folder: `mv /home/ubuntu/f11/ /home/ubuntu/test4/`

Step 2: Move nested folder: `mv /home/ubuntu/f1/f11/ /home/ubuntu/test4/`

■■ `mv` does not need `-r`. It moves/renames both files and folders.

7. rm / rmdir — Delete Files and Folders

Step 1: Delete files: `rm 1.txt 2.txt`

Step 2: Delete empty folder: `rmdir f1`

Step 3: Delete non-empty folder: `rm -r f2`

■■ `rm -r` is permanent. No recycle bin in Linux. Double-check the path!

8. apt-get — Install Software Packages

Step 1: Update package list first (always do this before install): `sudo apt-get update`

Step 2: Install a package: `sudo apt-get install packagename`

Step 3: Example — install Apache web server: `sudo apt-get install apache2`

■■ Always run `sudo apt-get update` BEFORE installing any package. Skipping causes errors.

9. grep — Search Text in Files

Step 1: Basic search: `grep 'word' filename.txt`

Step 2: Case insensitive: `grep -i 'word' filename.txt`

Step 3: Exact word match: `grep -w 'word' filename.txt`

Step 4: Case insensitive + exact: `grep -iw 'word' filename.txt`

Step 5: Exclude/omit pattern: `grep -v 'word' filename.txt`

Step 6: With pipeline: `ps -e | grep 'ssh'`

10. awk — Process Table/Column Data

Step 1: Print all data: `awk '{print $0}' reports.txt`

Step 2: Print first column: `awk '{print $1}' reports.txt`

Step 3: Print multiple columns: `awk '{print $2,$3}' reports.txt`

Step 4: Add row numbers: `awk '{print NR,$2,$3}' reports.txt`

Step 5: Count fields in each row: `awk '{print NF}' reports.txt`

Step 6: Print 2nd row only: `awk '{if (NR==2) print $0}' reports.txt`

Step 7: Print 4th column of 2nd row: `awk '{if (NR==2) print $4}' reports.txt`

■ `NR` = row number, `NF` = number of fields, `$0` = whole row, `$1..$N` = columns

11. Pipeline (|) — Chain Commands

Step 1: Pipe output of one command into next: `ps -e | grep 'ssh'`

Step 2: Example chain: `mkdir kal | touch kalyan`

12. ps — View Running Processes

Step 1: All background processes: `ps -e`

Step 2: Frontend processes: `ps -f`

Step 3: All frontend + backend: `ps -alx` or `ps -elx`

■ `PPID` = parent process ID, `STIME` = start time. These may be asked in viva.

13. alias — Create Command Shortcuts

Step 1: Temporary alias: `alias linux='sudo apt-get update'`

Step 2: To make it permanent — open `.bashrc`: `sudo su` then `nano ~/.bashrc`

Step 3: Add the alias line at bottom, save, then: `source ~/.bashrc`

14. man — Manual / Documentation

Step 1: View docs for any command: `man ls`

Step 2: Press `q` to quit the manual page

15. ping — Test Network Connectivity

Step 1: Ping Google: `ping google.com`

Step 2: Stop with `Ctrl+C`

■ **Exam Checklist:** Write theory → Open terminal → Run commands step by step → Show output to examiner → Never skip sudo apt-get update before install.

Experiment No. 2

Title: Implementation of Staging, Commit and Push on GitHub

Objectives

To perform Version Control using GIT — Staging, Commit and Push on GitHub.

Theory

Git is a free, open-source distributed version control system (VCS) used to track changes in source code. It allows multiple developers to collaborate efficiently. GitHub is a cloud-based platform that hosts Git repositories. The basic Git operations are: Initialize → Add (Stage) → Commit → Push/Pull. Advanced operations include Branching, Merging, and Rebasing.

Git Flow

Working Directory → git add (Stage) → git commit (Local Repo) → git push (GitHub Remote)

Installation of Git

Step 1: In Windows: Download from <https://git-scm.com> and install. Then open Git Bash.

Step 2: In Ubuntu: `sudo apt-get update` → `sudo apt install git`

Step 3: Confirm version: `git --version`

■ Always verify with `git --version` after install. Examiner will notice if you skip it.

Step 1 — Setup & Initialize

Step 1: Initialize a git repository: `git init`

Step 2: Clone repo from GitHub: `git clone <repository-url>`

Step 3: Move into the cloned folder: `cd <repository-name>`

■ Get the URL from GitHub → green 'Code' button → copy HTTPS link.

Step 2 — Staging (git add)

Step 1: Create a file: `touch index.html` OR `echo 'hello' > index.html`

Step 2: Stage a specific file: `git add index.html`

Step 3: Stage ALL files at once: `git add .`

Step 4: Check what is staged: `git status`

■ Files in green = staged. Files in red = not staged. Always check status after add.

Step 3 — Commit (git commit)

Step 1: Commit with a message: `git commit -m "Added source code for experiment 1"`

Step 2: Verify commit was saved: `git log`

■ Always write a meaningful message inside quotes. Empty message will cause an error.

■ You should see your commit hash, author, date, and message in git log.

Step 4 — Push to GitHub (git push)

Step 1: Push to master branch: `git push origin master`

Step 2: If branch is 'main' (newer GitHub default): `git push origin main`

Step 3: Enter your GitHub username and password/token when prompted

Step 4: Go to GitHub website and verify the file is visible in your repository

■ GitHub now uses Personal Access Tokens instead of passwords. If login fails, use a token from GitHub → Settings → Developer Settings → Personal Access Tokens.

Other Important Commands

Step 1: Check repository status: `git status`

Step 2: View commit history: `git log`

Step 3: Compact one-line log: `git log --oneline`

Step 4: Pull latest changes from remote: `git pull origin master`

Step 5: Undo changes to a file: `git checkout <file-name>`

Step 6: Create new branch: `git branch <branch-name>`

Step 7: Switch to a branch: `git checkout <branch-name>`

Step 8: Merge two branches: `git merge <branch-name>`

Complete Experiment Steps (Do exactly in this order)

Step 1: Install git and verify: `git --version`

Step 2: Create a GitHub account and new repository at github.com

Step 3: Clone repo: `git clone <your-repo-url>` then `cd <repository-name>`

Step 4: Create a new file: `echo 'print(Hello DevOps)' > hello.py`

Step 5: Check status: `git status` — file should appear in red

Step 6: Stage the file: `git add hello.py`

Step 7: Check status again: `git status` — file should now appear in green

Step 8: Commit changes: `git commit -m "Added hello.py for experiment 1"`

Step 9: Push to GitHub: `git push origin master`

Step 10: Open GitHub in browser → verify the file appears in your repository

■ **Exam Checklist:** Write theory → Install git → git init/clone → create file → git add → git status → git commit → git push → show GitHub repo to examiner.

Experiment No. 3

Title: Commit, Modification on Branch Master, Forking and GitHub Repository Link

Objectives

To implement Commit, Modification on Branch Master, Forking and GitHub Repository Link.

Theory

A Git branch is a separate version of the main repository that allows working on changes without affecting the master branch. When work is complete, branches are merged back. A Fork is a copy of someone else's GitHub repository into your own account — used to contribute to others' projects. Fork is not a Git command; it is a GitHub feature. Key advanced operations: Branching, Merging, Rebasing, Stashing, Cherry-pick, and Revert.

Branch Flow

master → git branch → checkout + commit → git push → git merge → git branch -d (delete)

1. Clone and View Branches

Step 1: Clone into current directory: `git clone 'URL-of-Remote-Repository'`

Step 2: Clone into specific folder: `git clone 'URL' desktop/gitrepo1`

Step 3: View local branches: `git branch`

Step 4: View all branches (local + remote): `git branch -a`

2. Create New Branches

Step 1: Create merge branch: `git branch mergebranch`

Step 2: Create rebase branch: `git branch rebasebranch`

■ Active branch is shown with a * star. You start on master by default.

3. Switch to Branch and Commit

Step 1: Switch to mergebranch: `git checkout mergebranch`

Step 2: Create a new file: `touch branch-file.txt`

Step 3: Stage and commit: `git add .` then `git commit -m 'Added file in mergebranch'`

Step 4: Push branch to remote: `git push origin mergebranch`

Step 5: Push ALL branches at once: `git push --all origin`

Step 6: Verify all branches: `git branch -a`

4. Merge Branches into Master

Step 1: Go to master first: `git checkout master`

Step 2: Pull latest changes: `git pull origin master`

Step 3: Check what's already merged: `git branch --merged`

Step 4: Merge the branch: `git merge mergebranch`

■■ Always be on the TARGET branch (master) before running merge. Not on the source branch.

5. Rebase Branches

Step 1: Switch to master: `git checkout master`

Step 2: Add a file and commit on master: `touch rebase-test.txt → git add . → git commit -m 'test'`

Step 3: Rebase with rebasebranch: `git rebase rebasebranch`

■ Rebase replays commits on top of another branch — keeps history clean and linear.

6. Delete a Branch

Step 1: Check merged branches: `git branch --merged`

Step 2: Delete local branch: `git branch -d mergebranch`

Step 3: Verify deletion: `git branch -a`

Step 4: Delete from remote (GitHub): `git push origin --delete mergebranch`

■ Only delete after merging. Use -d (safe). -D forces deletion even without merge.

7. Undo Changes

Step 1: Undo unstaged changes in a file: `git checkout filename.txt`

Step 2: Edit last commit message: `git commit --amend -m 'Updated Message'`

Step 3: Soft reset (keeps changes staged): `git reset --soft <commit-hash>`

Step 4: Mixed reset (moves changes to working dir): `git reset <commit-hash>`

Step 5: Hard reset (deletes all changes permanently): `git reset --hard <commit-hash>`

Step 6: Remove untracked files/folders: `git clean -df`

■ --hard reset is permanent. All uncommitted work is lost. Use with caution.

8. Cherry-pick — Move a Commit to Another Branch

Step 1: Switch to the target branch: `git checkout newbranch`

Step 2: Apply specific commit from another branch: `git cherry-pick <commit-hash>`

■ Get commit-hash using: `git log --oneline` before running cherry-pick.

9. Reflog — Recover Lost Commits

Step 1: See all recent actions: `git reflog`

Step 2: Checkout lost commit: `git checkout <commit-hash>`

Step 3: Create backup branch: `git branch backup`

Step 4: Switch back to master: `git checkout master`

■ Works only before garbage collector runs. Use this to rescue commits deleted by hard reset.

10. Revert a Commit (Safe Undo)

Step 1: Get commit hash from log: `git log --oneline`

Step 2: Revert that commit: `git revert <commit-hash>`

Step 3: Save and exit the message (Ctrl+X → Y → Enter in nano)

Step 4: Compare original vs reverted: `git diff <commit-hash> <revert-hash>`

■ Use revert (not reset) when others have already pulled your commits — it's safe for shared repos.

11. Stash — Save Work Without Committing

Step 1: Save current changes to stash: `git stash save 'Stash-message'`

Step 2: View all stashes: `git stash list`

Step 3: Restore stash but keep it in list: `git stash apply stash@{0}`

Step 4: Go back to state before apply: `git checkout -- .`

Step 5: Restore most recent stash and remove it: `git stash pop`

Step 6: Delete a specific stash: `git stash drop stash@{0}`

Step 7: Clear all stashes: `git stash clear`

■ stash apply keeps the stash in list. stash pop removes it after restoring.

12. Fork a GitHub Repository

Step 1: Go to the target repository on GitHub (github.com)

Step 2: Click the Fork button at the top-right of the page

Step 3: GitHub creates a copy under your own account

Step 4: Clone your fork locally: `git clone <your-fork-URL>`

Step 5: Make changes, commit, and push to your fork

Step 6: On GitHub, click 'New pull request' to propose changes to the original repo

■ Fork is a GitHub feature, NOT a Git command. It copies the entire repo into your GitHub account.

■■ You cannot directly push to someone else's repository without access rights — that's why fork exists.

Complete Experiment Steps (Do exactly in this order)

Step 1: Clone the repo: `git clone 'URL'` then `cd repo-name`

Step 2: View branches: `git branch -a`

Step 3: Create two branches: `git branch mergebranch` and `git branch rebasebranch`

Step 4: Switch to mergebranch: `git checkout mergebranch`

Step 5: Add file + commit: `touch merge-file.txt` → `git add .` → `git commit -m 'file in mergebranch'`

Step 6: Push branch to GitHub: `git push origin mergebranch`

Step 7: Go to master and merge: `git checkout master` → `git pull origin master` → `git merge mergebranch`

Step 8: Delete merged branch: `git branch -d mergebranch` → `git push origin --delete mergebranch`

Step 9: Demonstrate stash: make a change → `git stash save 'test stash'` → `git stash list` → `git stash pop`

Step 10: Show fork on GitHub: go to any public repo → click Fork → show copied repo in your account

■ **Exam Checklist:** Write theory → Clone repo → Create branches → Checkout + commit → Push → Merge into master → Delete branch → Show stash → Show fork on GitHub.

Experiment No. 4

Title: Implementation of CI/CD with Jenkins and Java & cloning in GitHub

Objectives

To implement Continuous Integration/Continuous Development with Jenkins and Java.

Theory

CI (Continuous Integration) is the practice of automatically building and testing code every time a developer commits to the repository. CD (Continuous Deployment/Delivery) is the practice of automatically deploying tested code to production. Jenkins is an open-source automation server that enables CI/CD pipelines. It integrates with Git/GitHub, builds Java applications, runs tests, and deploys — all automatically on every commit.

CI/CD Flow

Java Code → GitHub Repo → Jenkins Job → Build & Test → JAR File → Deploy

Step 1 — Install Java JDK

Step 1: Update packages: `sudo apt-get update`

Step 2: Install Java JDK: `sudo apt install openjdk-11-jdk`

Step 3: Verify Java: `java -version`

■ Jenkins requires Java to run. Always install Java first before Jenkins.

Step 2 — Install Jenkins on Ubuntu

Step 1: Add Jenkins key: `curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc`

Step 2: Add Jenkins repo: `echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list`

Step 3: Update and install: `sudo apt-get update` then `sudo apt-get install jenkins`

Step 4: Start Jenkins: `sudo systemctl start jenkins`

Step 5: Enable on boot: `sudo systemctl enable jenkins`

Step 6: Check status: `sudo systemctl status jenkins`

■ Jenkins runs on port 8080 by default. Open browser → `http://localhost:8080`

Step 3 — Unlock and Setup Jenkins UI

Step 1: Get initial admin password: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`

Step 2: Paste the password into Jenkins web UI at `http://localhost:8080`

Step 3: Click 'Install suggested plugins' and wait

Step 4: Create your admin username and password

Step 5: Click 'Save and Finish' → Jenkins is ready

■■ Copy the admin password before pasting. The terminal will show a long alphanumeric string — that's the password.

Step 4 — Create a Simple Java Application

Step 1: Create project folder: `mkdir JavaDemo` then `cd JavaDemo`

Step 2: Create Java file: `nano HelloWorld.java`

Type this code inside:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World - Jenkins CI/CD");  
    }  
}
```

```
}
```

```
}
```

Step 3: Save: Ctrl+X → Y → Enter

Step 4: Compile to verify: javac HelloWorld.java

Step 5: Run to verify output: java HelloWorld

■ You should see: Hello World - Jenkins CI/CD — this confirms Java works.

Step 5 — Push Java Code to GitHub

Step 1: Initialize git: git init

Step 2: Stage the Java file: git add HelloWorld.java

Step 3: Commit: git commit -m "Added HelloWorld Java app"

Step 4: Add remote GitHub repo: git remote add origin <your-repo-URL>

Step 5: Push to GitHub: git push origin master

Step 6: Verify file is visible on GitHub in your browser

■ ■ Make sure the GitHub repository is PUBLIC or Jenkins has access credentials configured.

Step 6 — Create a Jenkins Freestyle Job

Step 1: Open Jenkins: go to http://localhost:8080 in browser

Step 2: Click 'New Item' on the left sidebar

Step 3: Enter a job name e.g. JavaCICD-Job

Step 4: Select 'Freestyle project' and click OK

Step 7 — Connect Jenkins to GitHub Repo

Step 1: In job configuration → scroll to 'Source Code Management'

Step 2: Select 'Git'

Step 3: Paste your GitHub repository URL in the 'Repository URL' field

Step 4: Set branch to */master or */main

■ ■ If the repo is private, add credentials (GitHub username + personal access token) under Credentials.

Step 8 — Set Automatic Build Trigger

Step 1: Scroll to 'Build Triggers' section

Step 2: Check 'Poll SCM' to let Jenkins check GitHub periodically

Step 3: Enter schedule: * * * * * (checks every minute)

■ This means Jenkins will poll GitHub every minute. If new commit is found, it triggers a build automatically.

Step 9 — Add Build Steps (Compile Java)

Step 1: Scroll to 'Build Steps' section → click 'Add build step'

Step 2: Select 'Execute shell'

Step 3: Enter the compile command: javac HelloWorld.java

Step 4: Optionally add run command: java HelloWorld

Step 5: Click 'Save' at the bottom

■ Jenkins will now compile and run your Java file automatically on every build.

Step 10 — Trigger and Monitor the Build

Step 1: Go to your job dashboard in Jenkins

Step 2: Click 'Build Now' on the left sidebar

Step 3: Watch the build appear under 'Build History' at the bottom-left

Step 4: Click the build number (e.g. #1) → click 'Console Output'

Step 5: Verify you can see 'Hello World - Jenkins CI/CD' in the output

Step 6: Build status: Blue/Green = SUCCESS, Red = FAILED

■ Console Output is the most important thing to show the examiner — it proves the build ran successfully.

Step 11 — Demonstrate Full CI/CD Cycle

Step 1: Make a small change to HelloWorld.java: nano HelloWorld.java — change the print message

Step 2: Stage and commit: git add . → git commit -m 'Updated message'

Step 3: Push to GitHub: git push origin master

Step 4: Go back to Jenkins — within 1 minute a new build (#2) will trigger automatically

Step 5: Check Console Output of build #2 — it shows the new message

■ This demonstrates CI — Jenkins automatically detected the commit and rebuilt without any manual trigger.

Jenkins Service Commands

Step 1: Start Jenkins: sudo systemctl start jenkins

Step 2: Stop Jenkins: sudo systemctl stop jenkins

Step 3: Restart Jenkins: sudo systemctl restart jenkins

Step 4: Check status: sudo systemctl status jenkins

Step 5: Access UI: open browser → http://localhost:8080

Complete Experiment Steps (Do exactly in this order)

Step 1: Install Java: sudo apt install openjdk-11-jdk → verify: java -version

Step 2: Install Jenkins → start: sudo systemctl start jenkins → check: sudo systemctl status jenkins

Step 3: Open browser → http://localhost:8080 → unlock with: sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Step 4: Install suggested plugins → create admin user → Jenkins dashboard is ready

Step 5: Create Java file: nano HelloWorld.java → write Hello World code → compile → run

Step 6: Push Java code to GitHub: git init → git add . → git commit -m 'Java app' → git push origin master

Step 7: Jenkins → New Item → Freestyle project → add GitHub repo URL → set branch to master

Step 8: Build Triggers → Poll SCM → enter * * * * *

Step 9: Build Steps → Execute shell → enter javac HelloWorld.java and java HelloWorld → Save

Step 10: Click 'Build Now' → watch build #1 → open Console Output → show output to examiner

Step 11: Modify HelloWorld.java → commit → push → Jenkins auto-builds #2 (shows CI working)

■ **Exam Checklist:** Write theory → Install Java + Jenkins → Unlock Jenkins UI → Create Java app → Push to GitHub → Create Jenkins job → Link GitHub repo → Add build step → Build Now → Show Console Output → Demonstrate auto-build on commit.

Experiment No. 5

Title: Implementation of Docker Commands

Objectives

To implement Docker commands for container management.

Theory

Docker is an open-source containerization platform that packages applications and their dependencies into lightweight, portable units called containers. Unlike virtual machines, containers share the host OS kernel, making them fast and efficient. A Docker Image is a read-only template used to create containers. A Docker Container is a running instance of an image. Docker Hub is the public registry where images are stored and shared.

Container Flow

docker pull → docker run → docker ps → docker stop → docker rm

Install Docker on Ubuntu

Step 1: Update packages: `sudo apt-get update`

Step 2: Install Docker: `sudo apt-get install docker.io`

Step 3: Start Docker service: `sudo systemctl start docker`

Step 4: Enable on boot: `sudo systemctl enable docker`

Step 5: Check status: `sudo systemctl status docker`

Step 6: Verify version: `docker --version`

■ You should see something like: Docker version 24.x.x — show this to the examiner.

■ All docker commands need sudo unless you add your user to the docker group.

docker run — Run a New Container

Step 1: Basic run with interactive shell: `sudo docker run --name mycontainer -it ubuntu:16.04 /bin/bash`

Step 2: Run in detached (background) mode: `sudo docker run --name mycontainer -d ubuntu:16.04`

Step 3: Run with port mapping: `sudo docker run --name webserver -d -p 8080:80 nginx`

Step 4: Exit the container shell (stay running): Ctrl+P then Ctrl+Q

Step 5: Exit and stop container: type exit

■ Flags: -it = interactive terminal, -d = detached/background, -p = port mapping, --name = container name.

■ If image is not found locally, Docker auto-pulls it from Docker Hub before running.

docker ps — List Containers

Step 1: Show only running containers: `sudo docker ps`

Step 2: Show ALL containers (running + stopped): `sudo docker ps -a`

■ Always run `docker ps -a` to show examiner all containers — even stopped ones are visible.

docker start / stop — Start and Stop Containers

Step 1: Start a stopped container: `sudo docker start mycontainer`

Step 2: Stop a running container: `sudo docker stop mycontainer`

Step 3: Restart a container: `sudo docker restart mycontainer`

Step 4: Verify: `sudo docker ps -a`

■ stop sends a graceful SIGTERM signal. Use `docker kill mycontainer` for immediate stop.

docker rm — Remove Containers

Step 1: Remove a stopped container: `sudo docker rm mycontainer`

Step 2: Force remove a running container: `sudo docker rm -f mycontainer`

Step 3: Remove all stopped containers: `sudo docker container prune`

■■ Must stop container before removing it. Or use `-f` flag to force remove.

docker exec — Run Commands Inside a Running Container

Step 1: Open shell inside running container: `sudo docker exec -it mycontainer /bin/bash`

Step 2: Run a single command inside container: `sudo docker exec mycontainer ls /`

Step 3: Exit the container: type `exit`

■ `exec` is used to get into an already-running container. Use `run` only to start a new one.

docker pull — Pull Image from Docker Hub

Step 1: Pull Ubuntu 16.04 image: `sudo docker pull ubuntu:16.04`

Step 2: Pull latest Ubuntu: `sudo docker pull ubuntu`

Step 3: Pull Nginx web server: `sudo docker pull nginx`

Step 4: Verify image was downloaded: `sudo docker images`

■ Images are pulled from `hub.docker.com`. You can search any image there first.

docker images — List All Local Images

Step 1: List all images: `sudo docker images`

Step 2: Shows: REPOSITORY, TAG, IMAGE ID, CREATED, SIZE

■ Show this output to examiner to prove images are available locally.

docker push — Push Image to Docker Hub

Step 1: Login to Docker Hub first: `sudo docker login` — enter username and password

Step 2: Tag your image with your Docker Hub username: `sudo docker tag myimage yourusername/myimage`

Step 3: Push the image: `sudo docker push yourusername/myimage`

■■ Image name **MUST** start with your Docker Hub username before pushing. Otherwise push will fail.

docker rmi — Remove Images

Step 1: Remove a specific image: `sudo docker rmi ubuntu:16.04`

Step 2: Remove by image ID: `sudo docker rmi <image-id>`

Step 3: Remove all unused images: `sudo docker image prune`

■■ Cannot remove an image that has an existing container (even stopped). Remove the container first with `docker rm`.

Dockerfile — Create Custom Image

Step 1: Create a project folder: `mkdir myapp && cd myapp`

Step 2: Create Dockerfile: `nano Dockerfile`

Type this content inside:

```
FROM ubuntu:20.04
```

```
RUN apt-get update
```

```
RUN apt-get install -y python3
```

```
COPY . /app
```

```
WORKDIR /app
```

```
CMD ["python3", "--version"]
```

Step 3: Save: `Ctrl+X` → `Y` → `Enter`

Step 4: Build the image: `sudo docker build -t myapp:1.0 .`

Step 5: Verify image is created: `sudo docker images`

Step 6: Run the custom image: `sudo docker run myapp:1.0`

■■ The dot (.) at end of docker build means 'look for Dockerfile in current directory' — don't forget it.

Other Useful Commands

Step 1: Inspect a container: `sudo docker inspect mycontainer`

Step 2: View container logs: `sudo docker logs mycontainer`

Step 3: Follow live logs: `sudo docker logs -f mycontainer`

Step 4: Copy file FROM container to host: `sudo docker cp mycontainer:/file.txt /home/ubuntu/`

Step 5: Copy file TO container from host: `sudo docker cp /home/ubuntu/file.txt mycontainer:/`

Step 6: Docker system disk usage: `sudo docker system df`

Step 7: Clean up all unused resources: `sudo docker system prune`

Complete Experiment Steps (Do exactly in this order)

Step 1: Install Docker: `sudo apt-get update` → `sudo apt-get install docker.io` → `sudo systemctl start docker`

Step 2: Verify install: `docker --version` → show version to examiner

Step 3: Pull Ubuntu image: `sudo docker pull ubuntu:16.04`

Step 4: List images: `sudo docker images` → verify ubuntu is listed

Step 5: Run container with shell: `sudo docker run --name mycontainer -it ubuntu:16.04 /bin/bash`

Step 6: Inside container run: `ls` and `echo 'Hello from Docker'` → `exit`: `exit`

Step 7: List all containers: `sudo docker ps -a` → show mycontainer listed as Exited

Step 8: Start stopped container: `sudo docker start mycontainer` → verify: `sudo docker ps`

Step 9: Execute command in running container: `sudo docker exec -it mycontainer /bin/bash` → `exit`

Step 10: Stop container: `sudo docker stop mycontainer`

Step 11: Remove container: `sudo docker rm mycontainer` → verify: `sudo docker ps -a`

Step 12: Create Dockerfile → build custom image: `sudo docker build -t myapp:1.0 .` → run it

■ **Exam Checklist:** Write theory → Install Docker → `docker pull` → `docker images` → `docker run` (with -it) → `docker ps -a` → `docker start` → `docker exec` → `docker stop` → `docker rm` → Build Dockerfile image → Show all outputs to examiner.

Experiment No. 6

Title: Implementation of Containerization using Docker

Objectives

To implement Containerization using Docker — covering Dockerfile, Volumes, Networks, and Docker Compose.

Theory

Containerization is the process of packaging an application along with all its dependencies, libraries, and configuration files into a single portable unit called a container. Docker is the most widely used containerization platform. Unlike Virtual Machines, containers share the host OS kernel, making them lightweight and fast. Key components: Dockerfile (build instructions), Image (built template), Container (running instance), Volume (persistent storage), Network (container communication), and Docker Compose (multi-container management).

Containerization Flow

Dockerfile → docker build → Image → docker run → Container → docker push

1. Dockerfile — Write a Dockerfile for a Python App

Step 1: Create project folder: `mkdir myapp && cd myapp`

Step 2: Create app file: `nano app.py` → type: `print('Hello from Docker Container!')` → save

Step 3: Create Dockerfile: `nano Dockerfile` → type this content:

```
# Base image
FROM python:3.9-slim

# Set working directory inside container
WORKDIR /app

# Copy all files from host to container
COPY . /app

# Command to run when container starts
CMD ["python3", "app.py"]
```

Step 4: Save: `Ctrl+X` → `Y` → `Enter`

■ Dockerfile keywords examiner may ask: FROM, WORKDIR, COPY, RUN, CMD, EXPOSE, ENV.

2. Build Image from Dockerfile

Step 1: Build image (from current directory): `sudo docker build -t myapp:1.0 .`

Step 2: Build with different tag version: `sudo docker build -t myapp:2.0 .`

Step 3: List images to confirm: `sudo docker images`

Step 4: Run the built image: `sudo docker run myapp:1.0`

■ Output should print: Hello from Docker Container! — show this to examiner.

■ The dot (.) at end means 'use Dockerfile from current directory.' Never skip it.

3. Containerize a Web Application (Nginx)

Step 1: Create folder: `mkdir webapp && cd webapp`

Step 2: Create HTML file: `nano index.html` → type: `<h1>Hello DevOps Containerization!</h1>` → save

Step 3: Create Dockerfile: `nano Dockerfile`

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

Step 4: Build: `sudo docker build -t webapp:1.0 .`

Step 5: Run with port mapping: `sudo docker run -d --name webcontainer -p 8080:80 webapp:1.0`

Step 6: Open browser → go to `http://localhost:8080` → verify page shows

■ EXPOSE in Dockerfile documents the port. `-p 8080:80` actually maps it. Both needed for web apps.

4. Docker Volumes — Persistent Storage

Step 1: Create a named volume: `sudo docker volume create myvolume`

Step 2: List all volumes: `sudo docker volume ls`

Step 3: Inspect volume details: `sudo docker volume inspect myvolume`

Step 4: Run container with volume mounted: `sudo docker run -it --name volcontainer -v myvolume:/data ubuntu /bin/bash`

Step 5: Inside container create a file: `echo 'Persistent Data' > /data/test.txt` then exit

Step 6: Run a NEW container with same volume — data persists: `sudo docker run -it -v myvolume:/data ubuntu cat /data/test.txt`

Step 7: Delete a volume: `sudo docker volume rm myvolume`

■ Volumes persist data even after container is deleted. This is the key advantage over container storage.

■■ Cannot delete a volume that is being used by a container — stop and remove the container first.

5. Bind Mount — Mount Host Directory into Container

Step 1: Create folder on host: `mkdir /home/ubuntu/hostdata`

Step 2: Mount host folder into container: `sudo docker run -it -v /home/ubuntu/hostdata:/containerdata ubuntu /bin/bash`

Step 3: Any files created in `/containerdata` inside container appear in `/home/ubuntu/hostdata` on host

■ Bind mount = host path linked to container path. Volume = Docker-managed storage. Both persist data.

6. Docker Networks

Step 1: List all networks: `sudo docker network ls`

Step 2: Create a custom bridge network: `sudo docker network create mynetwork`

Step 3: Run container 1 on that network: `sudo docker run -d --name container1 --network mynetwork ubuntu sleep 3600`

Step 4: Run container 2 on same network: `sudo docker run -d --name container2 --network mynetwork ubuntu sleep 3600`

Step 5: Ping container1 from container2: `sudo docker exec container2 ping container1`

Step 6: Inspect network: `sudo docker network inspect mynetwork`

Step 7: Remove network: `sudo docker network rm mynetwork`

■ Default network types: bridge (default, single host), host (shares host network), none (isolated). Custom bridge networks allow containers to communicate by name.

7. Docker Compose — Multi-Container Management

Step 1: Install compose: `sudo apt-get install docker-compose`

Step 2: Verify version: `docker-compose --version`

Step 3: Create folder and file: `mkdir composeapp && cd composeapp → nano docker-compose.yml`

Step 4: Write this content:

```
version: '3'
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
    ports:
```

```
- '8080:80'
```

```
db:
```

```
image: mysql:5.7
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: password123
```

Step 5: Save: Ctrl+X → Y → Enter

Step 6: Start all containers: `sudo docker-compose up -d`

Step 7: Check running containers: `sudo docker-compose ps`

Step 8: View logs: `sudo docker-compose logs`

Step 9: Stop all: `sudo docker-compose down`

■ Docker Compose manages multi-container apps with one file. Examiner may ask why we use it — answer: to start/stop multiple containers with single command.

■ YAML is indentation-sensitive. Use spaces, NOT tabs. Wrong indent = error.

Dockerfile Keywords — All Important Instructions

Step 1: FROM — Base image to start from. e.g. FROM ubuntu:20.04

Step 2: RUN — Execute command during image build. e.g. RUN apt-get install -y python3

Step 3: COPY — Copy files from host to image. e.g. COPY . /app

Step 4: WORKDIR — Set working directory inside container. e.g. WORKDIR /app

Step 5: EXPOSE — Document which port the container listens on. e.g. EXPOSE 80

Step 6: ENV — Set environment variable. e.g. ENV APP_ENV=production

Step 7: CMD — Default command when container starts. e.g. CMD ['python3', 'app.py']

■ Difference between RUN and CMD — RUN executes at BUILD time, CMD executes at RUN time (when container starts).

VM vs Container (Most Asked Viva Question)

Step 1: Virtual Machine — Has full OS, heavy (GBs), slow to start (minutes), uses Hypervisor

Step 2: Container — Shares host OS kernel, lightweight (MBs), starts in seconds, uses Docker Engine

Step 3: Key advantage of containers — Portability: runs the same on any machine that has Docker

■ 'Works on my machine' problem is solved by containers — the environment travels with the app.

Complete Experiment Steps (Do exactly in this order)

Step 1: Install Docker: `sudo apt-get install docker.io` → start: `sudo systemctl start docker` → verify: `docker --version`

Step 2: Create project folder: `mkdir myapp && cd myapp`

Step 3: Create app.py: `echo 'print(Hello from Docker Container!)' > app.py`

Step 4: Create Dockerfile with FROM, WORKDIR, COPY, CMD instructions → save

Step 5: Build image: `sudo docker build -t myapp:1.0 .` → verify: `sudo docker images`

Step 6: Run container: `sudo docker run --name mycontainer myapp:1.0` → show output

Step 7: Create a volume: `sudo docker volume create myvolume` → attach to container → create file inside → exit

Step 8: Create network: `sudo docker network create mynetwork` → run 2 containers on it → ping between them

Step 9: Install compose: `sudo apt-get install docker-compose` → create docker-compose.yml with nginx + mysql

Step 10: Start compose: `sudo docker-compose up -d` → check: `sudo docker-compose ps` → open browser → `http://localhost:8080`

Step 11: Stop everything: `sudo docker-compose down` → cleanup: `sudo docker system prune`

■ **Exam Checklist:** Write theory → Create Dockerfile → `docker build` → `docker run` → show output → create volume → create network → ping between containers → write `docker-compose.yml` → `docker-compose up` → show running services in browser.

Objectives

To learn and demonstrate Maven Build Life Cycle.

Theory

Maven is a build automation and project management tool for Java projects. It uses a pom.xml (Project Object Model) file to define project configuration, dependencies, and plugins. Maven follows a predefined Build Lifecycle with sequential phases: clean → validate → compile → test → package → install → deploy. Developers only need to call one phase and Maven runs all previous phases automatically.

Key Maven Concepts

- **Project Object Model (POM):** The POM is an XML file named pom.xml that defines a project's configuration, dependencies, plugins, and goals.
- **Build Lifecycle:** Maven follows a predefined sequence of phases: clean, validate, compile, test, package, install, and deploy.
- **Plugin:** Extensions that provide specific functionality — compiling code, running tests, packaging artifacts.
- **Dependency Management:** Maven downloads dependencies from repositories like Maven Central.
- **Repository:** A collection of artifacts (JARs, etc.) that Maven uses. ~/.m2/repository is the local repo.

Maven Build Lifecycles

Clean Lifecycle: mvn clean — Deletes the target directory, removing all build artifacts.

Default Lifecycle: validate → compile → test → package → verify → install → deploy

Site Lifecycle: mvn site — Generates project documentation.

Step 1 — Install Java and Maven

Step 1: Update packages: `sudo apt-get update`

Step 2: Install Java JDK: `sudo apt install openjdk-11-jdk`

Step 3: Verify Java: `java -version`

Step 4: Install Maven: `sudo apt-get install maven`

Step 5: Verify Maven: `mvn -v`

■ You should see Maven version, Java version, and OS info. Show this to examiner.

Step 2 — Create Maven Project

Option A — Generate project using Maven archetype (easier and faster for exam):

Step 1: Auto-generate full Maven project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MavenDemo  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Step 2: Enter folder: `cd MavenDemo`

Step 3: View structure: `ls -R`

■ This auto-creates pom.xml, src/main/java, src/test/java and a sample App.java — best for exam speed.

Option B — Create manually:

Step 1: Create project folder structure: `mkdir -p MavenDemo/src/main/java`

Step 2: Enter project: `cd MavenDemo`

Step 3: Create Java file: `nano src/main/java/HelloWorld.java`

```
public class HelloWorld {
```

```
public static void main(String[] args) {  
    System.out.println("Hello, Maven!");  
}  
}
```

Step 4: Create pom.xml in project root: nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
    http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.example</groupId>  
  <artifactId>MavenDemo</artifactId>  
  <version>1.0-SNAPSHOT</version>  
</project>
```

■ groupId = organisation name, artifactId = project name, version = project version.

■■ pom.xml MUST be in the root of the project folder — same level as src/

Step 3 — Explore Maven Build Phases

Clean Phase:

Step 1: Run clean: mvn clean

Step 2: Deletes the target/ directory with all compiled files

■ Always run clean before a fresh build. Output: BUILD SUCCESS

Compile Phase:

Step 1: Compile Java source: mvn compile

Step 2: Compiled .class files appear in target/classes/

Step 3: Verify: ls target/classes/

■ This also runs validate phase automatically before compiling.

Test Phase:

Step 1: Run tests: mvn test

Step 2: Test results appear in target/surefire-reports/

■ Maven uses JUnit or TestNG automatically if declared in pom.xml dependencies.

Package Phase:

Step 1: Package the app: mvn package

Step 2: JAR file created at: target/MavenDemo-1.0-SNAPSHOT.jar

Step 3: Verify JAR exists: ls target/

Step 4: Run the JAR: java -cp target/MavenDemo-1.0-SNAPSHOT.jar HelloWorld

■ Output should print: Hello, Maven! — show this to examiner.

Install Phase:

Step 1: Install: mvn install

Step 2: JAR is copied to local Maven repo at: ~/.m2/repository/com/example/MavenDemo/

Step 3: Verify: ls ~/.m2/repository/com/example/MavenDemo/1.0-SNAPSHOT/

■ After install, other Maven projects on the same machine can use this as a dependency.

Combined Command:

Step 1: Clean + full build: mvn clean install

■ This is the most commonly used Maven command in real projects.

pom.xml Key Elements (Viva)

Step 1: groupId — Organisation/company identifier e.g. com.example

Step 2: artifactId — Project/module name e.g. MavenDemo

Step 3: version — Project version e.g. 1.0-SNAPSHOT (SNAPSHOT = in development)

Step 4: dependencies — External libraries the project needs (downloaded from Maven Central)

Step 5: plugins — Tools that add extra functionality to the build process

■■ When you run `mvn package`, Maven automatically runs `validate` + `compile` + `test` first. You don't need to run them separately.

Complete Experiment Steps (Do exactly in this order)

Step 1: Install Java: `sudo apt install openjdk-11-jdk` → verify: `java -version`

Step 2: Install Maven: `sudo apt-get install maven` → verify: `mvn -v`

Step 3: Generate project: `mvn archetype:generate ...` → `cd MavenDemo`

Step 4: View pom.xml: `cat pom.xml`

Step 5: Clean: `mvn clean` → output: BUILD SUCCESS

Step 6: Compile: `mvn compile` → verify: `ls target/classes/`

Step 7: Test: `mvn test` → output shows tests run

Step 8: Package: `mvn package` → verify JAR: `ls target/`

Step 9: Run JAR: `java -cp target/MavenDemo-1.0-SNAPSHOT.jar com.example.App` → output: Hello World!

Step 10: Install: `mvn install` → verify in local repo: `ls ~/.m2/repository/com/example/`

■ **Exam Checklist:** Write theory → Install Java + Maven → Generate project → Show pom.xml → Run clean → compile → test → package → Run JAR → install → show ~/.m2 repo to examiner.

Experiment No. 8

Title: Install Selenium Tool and Demonstration of Automated Testing using Selenium

Objectives

To learn automated testing using Selenium WebDriver.

Theory

Selenium is an open-source automated testing framework for web applications that supports multiple programming languages like Java and Python. Its three components are: (1) WebDriver — programmatic browser control using code, (2) Selenium IDE — record and playback without coding, and (3) Selenium Grid — parallel testing across multiple browsers/machines. WebDriver uses browser-specific drivers like ChromeDriver to communicate with the browser.

Components of Selenium

- Selenium WebDriver: Core component that provides a programming interface for creating and executing test scripts. Controls web browsers programmatically.
- Selenium IDE: Browser extension for recording and playback of user interactions. Simple graphical interface — no coding required.
- Selenium Grid: Distributed testing framework that allows running tests in parallel across multiple browsers, operating systems, and machines.

Selenium Test Flow

Test Script → WebDriver → ChromeDriver → Chrome Browser → Web App → Test Result

Step 1 — Install Java JDK

Step 1: Install JDK: `sudo apt install openjdk-11-jdk`

Step 2: Verify: `java -version`

Step 2 — Install Chrome and ChromeDriver

Step 1: Install Google Chrome: `sudo apt install google-chrome-stable`

Step 2: Check Chrome version: `google-chrome --version`

Step 3: Install ChromeDriver (matching version): `sudo apt install chromium-chromedriver`

Step 4: Verify ChromeDriver: `chromedriver --version`

Step 5: Check PATH location: `which chromedriver`

■■ Chrome version and ChromeDriver version MUST match. Mismatched versions = test will fail to launch.

■ ChromeDriver acts as a bridge between Selenium test script and the Chrome browser.

Step 3 — Install Maven

Step 1: Install Maven: `sudo apt-get install maven`

Step 2: Verify: `mvn -v`

Step 4 — Create Maven Project for Selenium

Step 1: Generate Maven project:

```
mvn archetype:generate -DgroupId=com.test -DartifactId=SeleniumDemo  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Step 2: Enter project: `cd SeleniumDemo`

Step 3: Open pom.xml: `nano pom.xml`

Step 5 — Add Selenium Dependency to pom.xml

Step 1: Add inside <dependencies> section of pom.xml:

```

<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.10.0</version>
  </dependency>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

Step 2: Save pom.xml → Maven will auto-download Selenium JAR on next build

Step 3: Download dependencies: mvn dependency:resolve

■ No manual JAR download needed — Maven fetches Selenium from Maven Central automatically.

Step 6 — Write Basic Selenium Test (Open Browser)

Step 1: Create test file: nano src/test/java/com/test/OpenBrowserTest.java

```

package com.test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class OpenBrowserTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver",
            "/usr/bin/chromedriver");
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com");
        System.out.println("Page Title: " + driver.getTitle());
        driver.quit();
    }
}

```

Step 2: Save → compile: mvn compile

Step 3: Run: mvn exec:java -Dexec.mainClass="com.test.OpenBrowserTest"

■ Chrome browser will open, go to Google, print title in terminal, then close. Show this to examiner.

■■ If running on a headless server (no display), add: options.addArguments('--headless') to ChromeOptions.

Common Selenium WebDriver Methods

Step 1: Open URL: driver.get("https://example.com")

Step 2: Get page title: driver.getTitle()

Step 3: Find element by ID: driver.findElement(By.id("username"))

Step 4: Find by name: driver.findElement(By.name("password"))

Step 5: Type into a field: .sendKeys("mytext")

Step 6: Click a button: .click()

Step 7: Close current tab: driver.close()

Step 8: Close all windows: driver.quit()

■ By class has many locators: By.id, By.name, By.xpath, By.cssSelector, By.linkText — examiner may ask these.

Step 7 — Write Login Automation Test (Main Experiment Script)

Step 1: Create file: nano src/test/java/com/test/LoginTest.java

```
package com.test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginTest {
    public static void main(String[] args)
        throws InterruptedException {
        System.setProperty("webdriver.chrome.driver",
            "/usr/bin/chromedriver");
        WebDriver driver = new ChromeDriver();
        driver.get("https://the-internet.herokuapp.com/login");
        System.out.println("Title: " + driver.getTitle());
        WebElement username =
            driver.findElement(By.id("username"));
        username.sendKeys("tomsmith");
        WebElement password =
            driver.findElement(By.id("password"));
        password.sendKeys("SuperSecretPassword!");
        WebElement loginBtn =
            driver.findElement(By.cssSelector("button[type='submit']"));
        loginBtn.click();
        Thread.sleep(2000);
        System.out.println("After login: " + driver.getTitle());
        System.out.println("Login Test PASSED!");
        driver.quit();
    }
}
```

Step 2: Save → compile: mvn compile

Step 3: Run test: mvn exec:java -Dexec.mainClass="com.test.LoginTest"

■ Browser opens → goes to login page → enters credentials automatically → clicks login → prints Login Test PASSED!

■ The demo site the-internet.herokuapp.com/login is a public practice site — credentials: tomsmith / SuperSecretPassword!

WebElement Locator Types (Viva)

Step 1: By.id — Fastest and most reliable: By.id("username")

Step 2: By.name — Use HTML name attribute: By.name("email")

Step 3: By.xpath — Flexible path: By.xpath("//input[@type='submit']")

Step 4: By.cssSelector — CSS style selector: By.cssSelector("button[type='submit']")

Step 5: By.linkText — Exact anchor link text: By.linkText("Click here")

■ Always prefer By.id when possible — fastest and least likely to break when page changes.

Complete Experiment Steps (Do exactly in this order)

Step 1: Install Java: `sudo apt install openjdk-11-jdk` → verify: `java -version`

Step 2: Install Chrome: `sudo apt install google-chrome-stable` → check version: `google-chrome --version`

Step 3: Install ChromeDriver: `sudo apt install chromium-chromedriver` → verify: `chromedriver --version`

Step 4: Install Maven: `sudo apt-get install maven` → verify: `mvn -v`

Step 5: Create Maven project: `mvn archetype:generate ...` → `cd SeleniumDemo`

Step 6: Edit `pom.xml` → add Selenium and JUnit dependencies → save → `mvn dependency:resolve`

Step 7: Write `OpenBrowserTest.java` → compile → run → Chrome opens Google → title prints in terminal

Step 8: Write `LoginTest.java` → compile → run → Chrome auto-fills credentials → clicks login → Login Test PASSED prints

Step 9: Show full console output to examiner — this proves automated testing is working

■ **Exam Checklist:** Write theory (3 components) → Install Java + Chrome + ChromeDriver + Maven → Create Maven project → Add Selenium dependency in `pom.xml` → Write basic open-browser test → Write login automation test → Run and show console output to examiner.